arXiv:2510.09312v2 [cs.CL] 22 Feb 2026

Verifying Chain-of-Thought Reasoning via Its Computational Graph

Zheng Zhao[1,3,*], **Yeskendir Koishekenov**[1], **Xianjun Yang**[2], **Naila Murray**[1], **Nicola Cancedda**[1]

[1]FAIR at Meta, [2]Meta Superintelligence Labs, [3]University of Edinburgh
[*]Work done during an internship at FAIR

Current Chain-of-Thought (CoT) verification methods predict reasoning correctness based on outputs (black-box) or activations (gray-box), but offer limited insight into *why* a computation fails. We introduce a white-box method: **Circuit-based Reasoning Verification (CRV)**. We hypothesize that attribution graphs of correct CoT steps, viewed as *execution traces* of the model's latent reasoning circuits, possess distinct structural fingerprints from those of incorrect steps. By training a classifier on structural features of these graphs, we show that these traces contain a powerful signal of reasoning errors. Our white-box approach yields novel scientific insights unattainable by other methods. (1) We demonstrate that structural signatures of error are highly predictive, establishing the viability of verifying reasoning directly via its computational graph. (2) We find these signatures to be highly domain-specific, revealing that failures in different reasoning tasks manifest as distinct computational patterns. (3) We provide evidence that these signatures are not merely correlational; by using our analysis to guide targeted interventions on individual transcoder features, we successfully correct the model's faulty reasoning. Our work shows that, by scrutinizing a model's computational process, we can move from simple error detection to a deeper, causal understanding of LLM reasoning.

Date: February 24, 2026
Correspondence: zheng.zhao@ed.ac.uk; canceddan@acm.org
Code: https://github.com/facebookresearch/CRV

Meta

1 Introduction

Chain-of-Thought (CoT; Wei et al., 2022; Kojima et al., 2022) prompting has proven to be a powerful method for boosting the performance of Large Language Models (LLMs). This capability is now central to the latest generation of reasoning models, such as DeepSeek-R1 (DeepSeek-AI et al., 2025) and OpenAI's o1 (OpenAI et al., 2024). Despite this success, a fundamental vulnerability persists across the spectrum of these systems: the reasoning process itself is sometimes flawed (Turpin et al., 2023; Li et al., 2025b; Arcuschin et al., 2025; Lindsey et al., 2025; Chen et al., 2025b).

This reliability gap has spurred research into automated verification. Current methods fall into two main categories. *Black-box* approaches analyze the generated text or final logit distribution (Jacovi et al., 2024; Wang et al., 2025b; Baker et al., 2025). *Gray-box* approaches look at the model's internal state, using simple probes on raw activations or analyzing the trajectory of hidden states (Xie et al., 2025; Zhang et al., 2025; Afzal et al., 2025; Bi et al., 2025; Wang et al., 2025a). While insightful, these methods are fundamentally limited; they can *detect* that a model's internal state is correlated with an error, but not *explain why* the underlying computation leads to an error.

This limitation motivates a deeper, more mechanistic approach. We postulate that models implement latent algorithms that solve specific tasks through specialized subgraphs, or **circuits** (Olah et al., 2020; Elhage et al., 2021). From this perspective, a reasoning failure is not merely an erroneous state, but a flaw in the execution of a latent algorithm. To diagnose such flaws requires inspecting the underlying computational process, akin to examining an *execution trace* in classical software. We propose to approximate this trace by constructing an **attribution graph** (Dunefsky et al., 2025)—a structural representation of the causal information flow between model components.

For such a graph to serve as a meaningful trace, its components must be interpretable. We therefore first create an interpretable surrogate model by replacing its standard MLP modules with trained **transcoders** (Dunefsky et al., 2025). We then construct and analyze attribution graphs over the sparsely activating features of such surrogate model (Ameisen et al., 2025). Finally, to formally test whether these traces contain a detectable signal of error, we train a diagnostic

classifier on their structural properties. This entire methodology, which we call **Circuit-based Reasoning Verification (CRV)**, is thus designed as a scientific instrument to investigate our central hypothesis: *that reasoning failures manifest as detectable structural signatures on their computational execution traces, which can be leveraged for automated verification.*

As a scientific instrument, CRV requires a controlled experimental setting. While advanced reasoning models employ complex mechanisms like search and backtracking, their convoluted reasoning paths can obscure the fundamental computations of a single reasoning step. Our work therefore focuses on standard, instruction-tuned models generating autoregressive CoT, as this paradigm provides a clearer window into the primitive computations that underpin emergent reasoning. While our approach, despite being effective, is too computationally intensive to be intended as a practical, drop-in verifier, it yields novel scientific insights unattainable by other methods. Our main contributions are therefore not just about performance, but about understanding:

- We introduce Circuit-based Reasoning Verification, a white-box method for analyzing reasoning failures, showing that verifying reasoning via its computational graph is feasible.
- We find that the structural signatures of error are highly domain-specific, revealing that failures in executing different reasoning tasks manifest as distinct computational patterns.
- We establish the causal role of these error signatures, successfully correcting faulty reasoning via targeted interventions on individual transcoder features.
- To support future research, we release datasets with step-level correctness labels for CoT reasoning on synthetic and real-world tasks, along with our trained transcoders.[1]

2 Problem Formulation and Preliminaries

2.1 Problem Statement

Let an LLM generate a Chain-of-Thought $S = (s_1, s_2, \ldots, s_m)$ to solve a problem, where each step s_i is a sequence of tokens. During the generation of step s_i, the underlying model produces a computational state $\mathcal{M}_i$. From this state, we construct an **attribution graph** $G_i = (\mathcal{V}, \mathcal{E})$, where vertices $\mathcal{V}$ represent interpretable features and tokens, and edges $\mathcal{E}$ represent the causal influence between them (see Section 3.2). From each graph G_i, we extract a fixed-size feature vector $\mathbf{x}_i = \phi(G_i)$, where ϕ is a feature extraction function designed to capture the graph's structural properties. We term this vector the step's *structural fingerprint*. Our goal is to learn a diagnostic classifier f_θ that takes this structural fingerprint as input to predict the correctness of the reasoning step:

$$\hat{y}_i = f_\theta(\mathbf{x}_i)$$

where $\hat{y}_i \in \{\text{correct}, \text{incorrect}\}$.

2.2 Preliminaries: Circuits in Transformers

The term "circuit" in mechanistic interpretability refers to a specific subgraph within a neural network that implements a human-understandable algorithm (Olah et al., 2020). In Transformers (Vaswani et al., 2017), these circuits are composed of attention heads and MLP computations. Our work is conceptually motivated by the prospect of finding patterns distinguishing sound and faulty activations of circuits involved in reasoning. While our method does not observe these circuits directly, our hypothesis is that they cast detectable *structural fingerprints* onto the attribution graphs we construct. A primary goal of our subsequent analysis is therefore to interpret the graph-based features that are most predictive of failure as the signatures of these underlying error patterns.

2.3 Preliminaries: Transcoders for Interpretable Features

A significant challenge in analyzing model activations is their high dimensionality and lack of direct interpretability. A powerful approach to this challenge is to learn a sparse, overcomplete basis for these activations using a sparse autoencoder (SAE; Cunningham et al., 2023). An SAE is trained to reconstruct an activation vector $x \in \mathbb{R}^d$ from a much

[1] https://github.com/facebookresearch/CRV

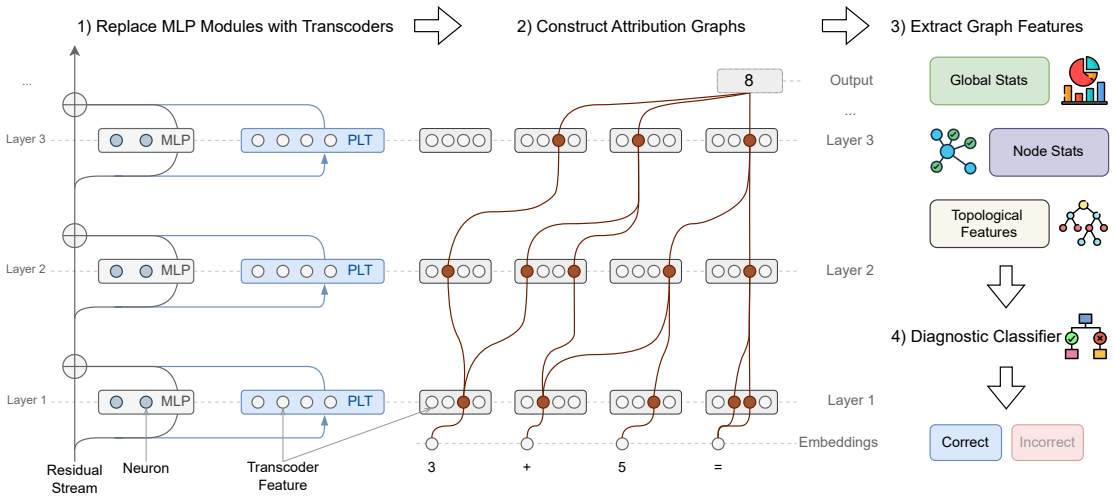

Figure 1 The CRV pipeline. (1) The LLM's MLP modules are replaced with per-layer transcoders (PLTs), making it interpretable. (2) For a given CoT step, we generate an attribution graph capturing causal flow between interpretable features and model components. (3) Structural features are extracted from this graph, and (4) fed to a diagnostic classifier to predict the step's correctness.

higher-dimensional, but mostly zero, feature vector $f \in \mathbb{R}^D$, where $D \gg d$. The elements of f correspond to a set of learned, interpretable features, sparsely activated by inputs. While the canonical SAE objective is to reconstruct its own input ($f(x) \approx x$), our work leverages a variant known as a transcoder (Dunefsky et al., 2025), which is instead trained to approximate the *input-output function* of a target component, such as an MLP ($f(x) \approx \text{MLP}(x)$). This approach makes the transcoder a true *functional substitute* for the original module. Its objective is not mere reconstruction, but the emulation of a computational step in an interpretable, sparsely activated basis. By replacing a model's standard MLP module with a trained transcoder, we force its intermediate computations to be represented not by a dense vector, but by a sparse combination of these meaningful features.

3 Methodology

Unlike in Process Reward Modeling (PRM), where the goal is limited to judging the correctness of a reasoning step, we take the perspective of a model developer interested in debugging reasoning failures in a specific model to which they have full access. We introduce **Circuit-based Reasoning Verification (CRV)**, a method for detecting flawed reasoning by analyzing its structural fingerprint.

3.1 Dataset Curation and Step-Level Annotation

A prerequisite for developing our method is a dataset with reliable step-level correctness labels. Furthermore, our white-box methodology imposes a critical requirement that distinguishes our data needs from prior work. Since CRV analyzes the causal computational graph that produces a reasoning step, we must capture the full internal state of our specific model during the generation process. Consequently, existing text-only datasets such as PRM800K (Lightman et al., 2024) and REVEAL (Jacovi et al., 2024), which provide static '(text, label)' pairs and are designed for training black-box verifiers, are incompatible with our mechanistic approach. We must generate and label our own model's CoT outputs to create the necessary '(text, label, computational trace)' tuples for analysis. We therefore created a new benchmark covering both controlled synthetic tasks and the real-world GSM8K dataset (Cobbe et al., 2021).

Synthetic Datasets (Boolean and Arithmetic). To study reasoning failures in a controlled environment, we generated two datasets. The first involves evaluating complex boolean expressions, while the second involves multi-step arithmetic problems. The motivation for these datasets is the unambiguous ground truth: the correctness of any step in the reasoning chain (e.g., "15 + 7 = 22") can be verified automatically by a simple parser and evaluator. This allows us to generate a large, labeled dataset for initial training and analysis. Furthermore, these tasks are intrinsically compositional, and the complexity of samples can be fully controlled. Further details are provided in Appendix A.